POKEMON CATH’EM ALL

Entities

Pokeball (Player), the Pokémon (Collectible), the Enemy AI (Enemy Pokemon), and the central Game Manager.

Scripts

The Pokeball uses the PokeballController.cs script for player movement and the CollectorLogic.cs script to handle collectable (pokemons) interactions, primarily detecting collisions using OnTriggerEnter().

Collectible **Pokémon** are defined by the Pokemon.cs script, which stores **OOP variables** like defense values, and contains the function to self-destruct upon collection.

The **Enemy AI** uses the script to utilize Unity's NavMeshAgent for constant tracking of the player.

All high-level game logic is handled by the **GameManager**, which uses **Arrays(???)** to spawn Pokémon, tracks the DefenseScore, and controls the final WinGame() or GameOver() **Functions**.

The primary interactions occur when the Pokeball's CollectorLogic detects a collision, calling the GameManager to update the score and the Pokemon script to destroy the object; similarly, AI collisions and reaching the **Finish Line** (identified by a Tag) rely on the CollectorLogic communicating these events directly to the GameManager.

Environment & Visual Setup

Feature	Task / Technical Requirement
Atmosphere & Lighting	By adjusting the scene Lighting settings (directional light). Setting up a suitable Skybox (e.g., bright forest) to catch the feeling.
Obstacles	Create and position 3D models (wall, tree, rock). Ensure these objects have Colliders (e.g., BoxCollider, MeshCollider)
Finish Line	Create a designated Finish Line object (e.g., a large ring, portal). The object must have a Collider set to Is Trigger and be given the Tag: FinishLine.
Background Sound	Import an audio track. Create an AudioSource component on an empty GameObject or the GameManager. Set it to Loop and Play On Awake.

Core Gameplay & Collection System

Script	Responsibility
GameManager.cs	**Update CurrentScore and DefenseScore.**
Pokemon.cs	**Assign the score value this specific Pokémon is worth. Now, it must also provide the *defense value* upon collection.**
CollectorLogic.cs	• Revising PlayerController script to make PickUpGameObjects disappear when they collide with the Player sphere • Tagging PickUp GameObjects and write conditional statements to make sure they're the only things that disappear • Setting the Prefab PickUp Collider as a trigger and add a Rigidbody component, to make the collectibles(pokemons) work properly
Collectables (Pokémon) Spawning(notsure)	float y = Random.Range(LowestPoint, HighestPoint); Vector3 position = new Vector3(x, y);
Enemy AI (Pokémon)	**Pathfinding:** Uses the **NavMeshAgent** component to find the shortest path to the player's Pokeball. **Tracking:** In Update() or FixedUpdate(), set the NavMeshAgent.SetDestination() to the Pokeball's current position.

Losing the game **Trigger**	**Condition (In AIPursuer.cs or CollectorLogic.cs)**	**Action (In GameManager.cs)**
Collision: AI Pokémon collides with Pokeball (using OnTriggerEnter).	**IF** GameManager.DefenseScore > 0	**Action:** Decrase DefenseScore.
Collision: AI Pokémon collides with Pokeball (using OnTriggerEnter).	**IF** GameManager.DefenseScore <= 0	**Action:** Call **GameOver()** function (Player Loses).

Winning the game

The player successfully reaches the Finish Line.

- **Trigger: Pokeball collides with the object tagged FinishLine.**
- **Action: Call WinGame() function in GameManager.**